CO1035: Game Concepts
Lab 1A: Introduction to C++ Programing Language

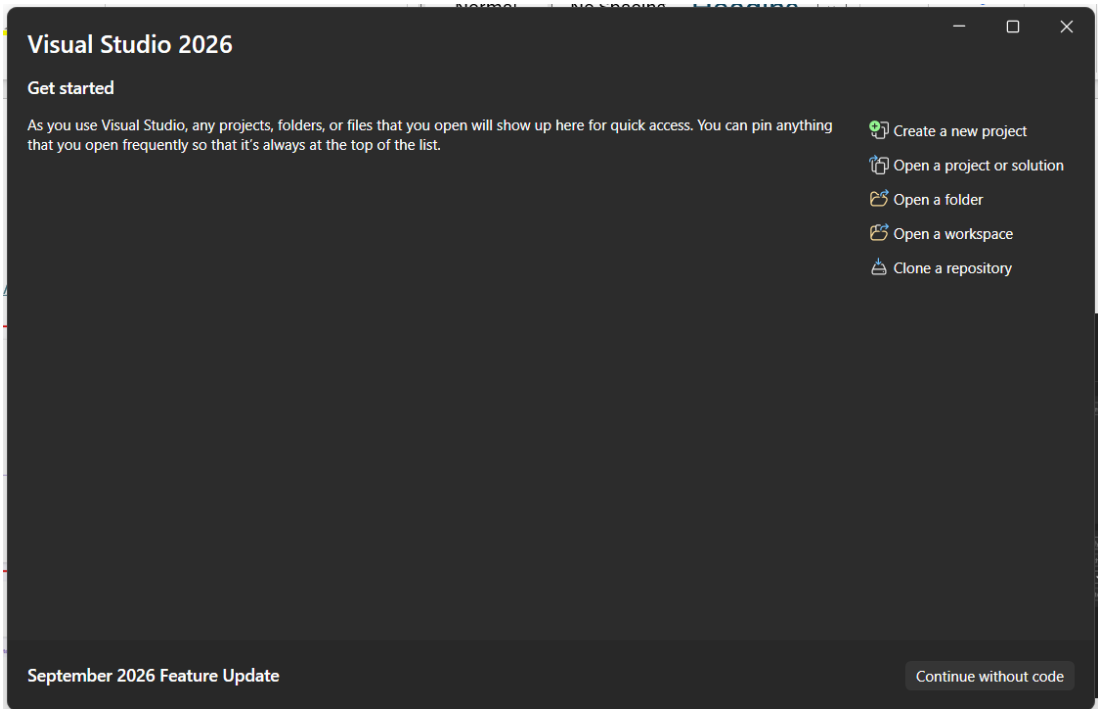
Overview:

This lab introduces the fundamental building blocks of C++ programming. You will work through progressively more challenging exercises covering input/output, decisions, loops, arrays, functions, and structs. Each topic begins with a concise explanation and further reading before you attempt the exercises. However, proficiency is not developed just by solving few exercises. Those are here to demonstrate how concepts that you are already familiar with are written in C++. For that reason, it is recommended to search for additional exercises and solve different problems.

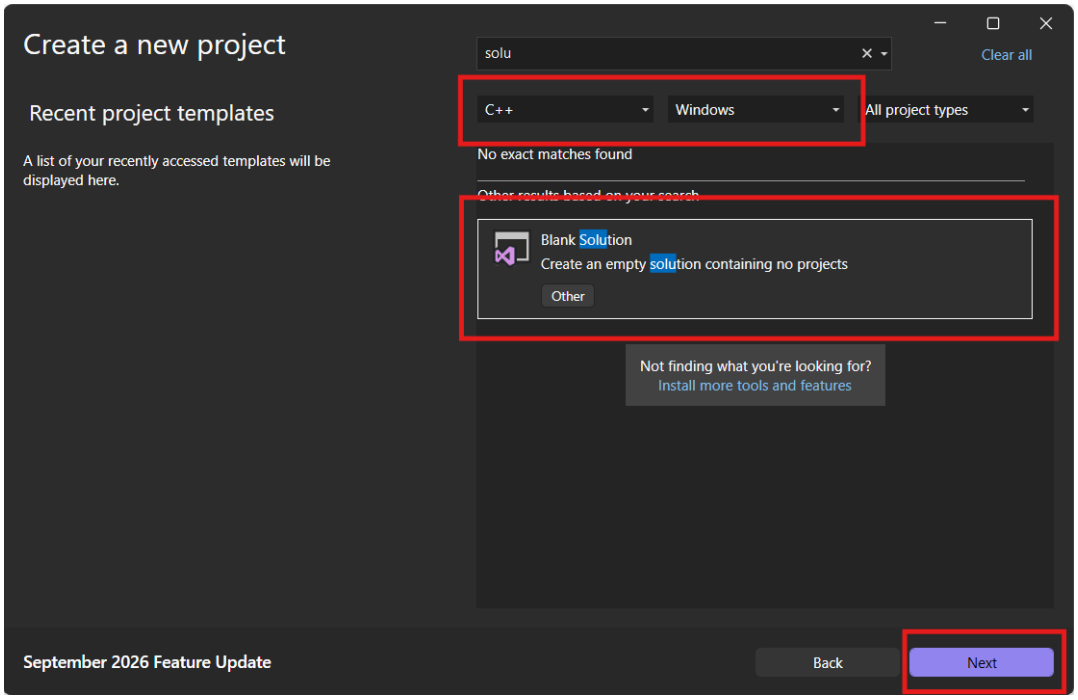
Setting up the Programming Environment

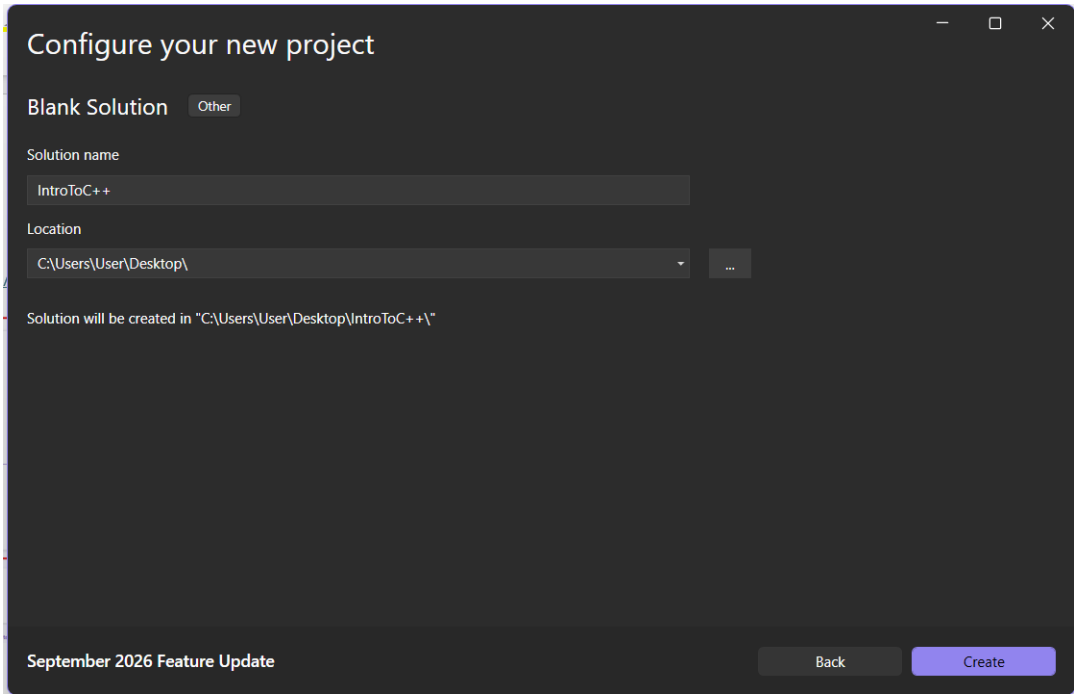
To set up the programming environment, it is assumed that you have already installed C++ and an IDE on your device. For this module, we will use Visual Studio. If you are in a computer lab, those are already installed. If you didn't manage to install Visual Studio on your computer, please follow the corresponding tutorial on Blackboard.

Once VS is downloaded, open it. Then, in the first window, click on "Create new Project".

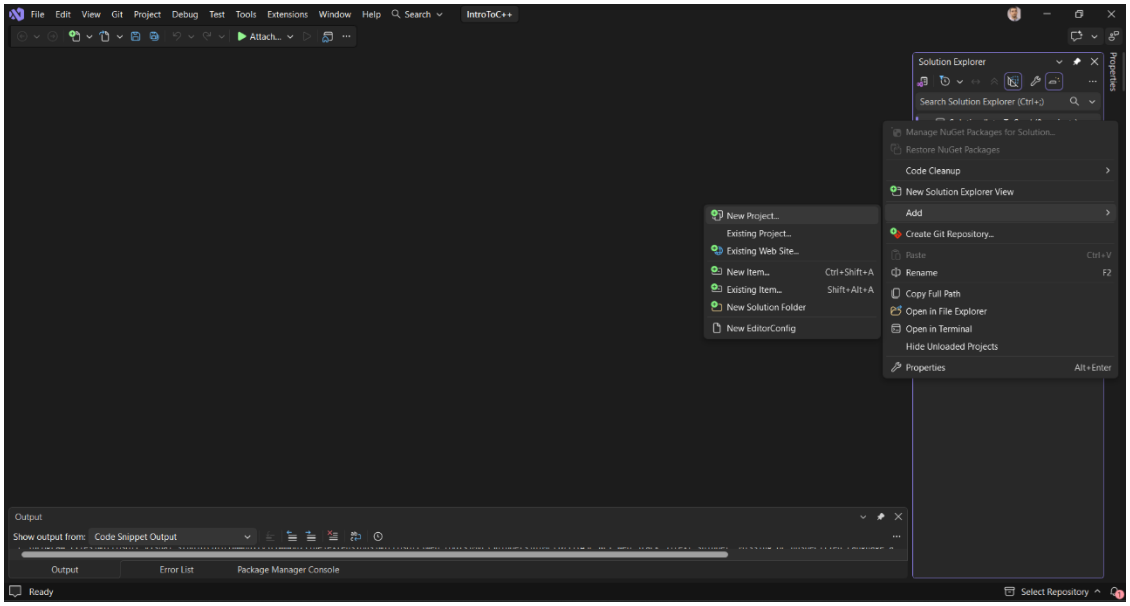


Then create a “Blank solution”. This will create one folder where subprojects can be stored. Give the solution a name, and place it somewhere you can easily access it.

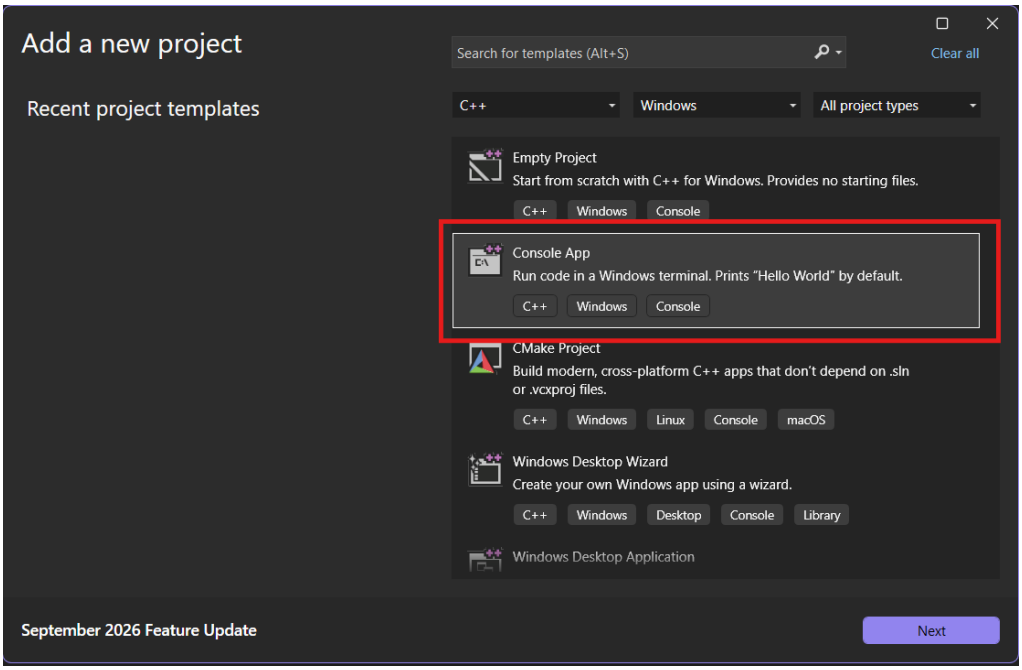




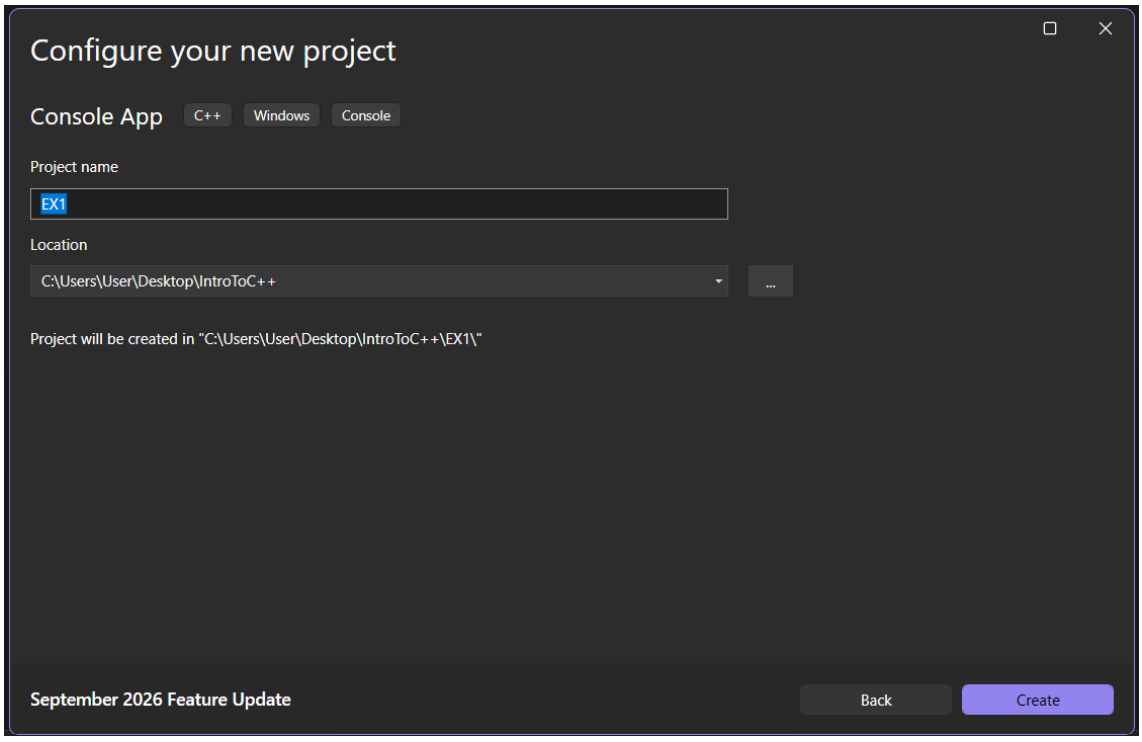
Once you have created the empty solution, in the Solution Explorer, right-click, then press on “Add” and then on “New Project”. This will create a subproject inside your solution. You will repeat the same process for every of the exercises below. If you cannot find the Solution Explorer, then press on “View” on the top task bar and then click on the “Solution Explorer” option.



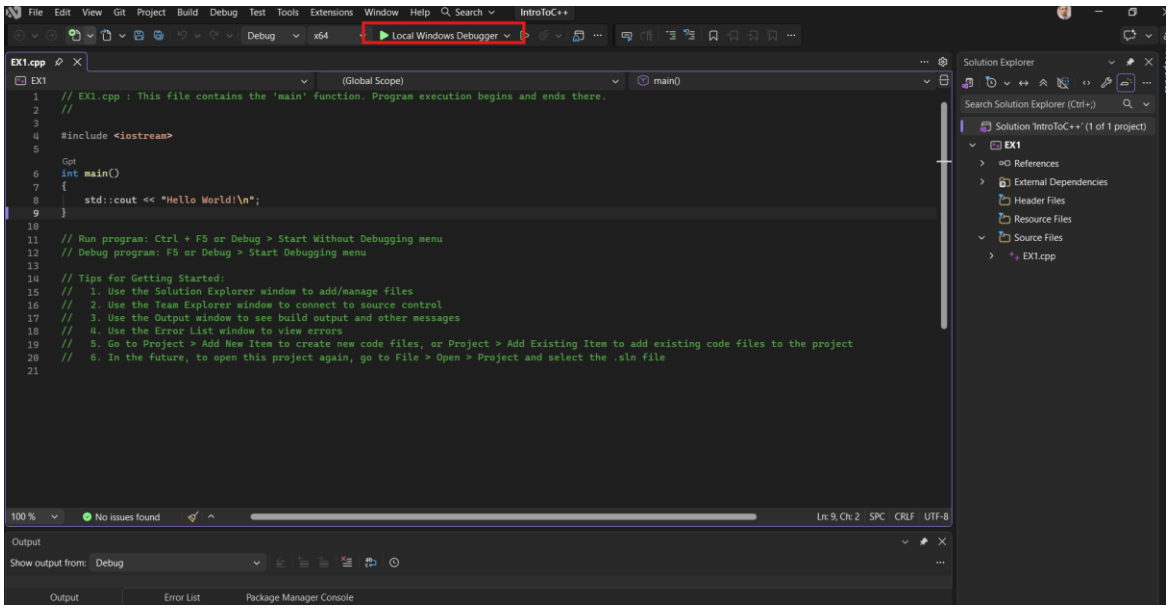
Once you press on “New Project”, a new window will appear. From that window, select the “Console App”. Make sure the console app is created for C++ and Windows. Then press next.



Then give your project a name (e.g., Exercise 1) and press the “Create” button. This will create one new empty project inside your solution.



This project can be found and accessed in the “Solution Explorer”. Once the project is created, a source code (a .cpp file) will appear on your screen, that includes a main function and some comments. To run the code, press on the “Local Windows Debugger” at the top.



Now repeat the same process again to create another project. Once you create a new project, if you want to run the code inside that new project, you need to specifically tell the editor which project to run. To do that, on the “Solution Explorer”, right-click on the project you want to run and then press “Set as startup Project”. This will make the project appear in Bold letters in your Solution Explorer.

Topic 1: Understanding the main

In C++, the main() function is the starting point of every executable program. When a C++ program is run, the computer begins executing the instructions written inside main().

A basic main() function looks like this:

```
int main()
{
    // Program instructions go here

    return 0;
}
```

The keyword int indicates that the function returns an integer value. The statement return 0; normally indicates that the program finished successfully.

For example:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello, World!" << endl;

    return 0;
}
```

In this example, execution begins inside main(), the message **"Hello, World!"** is displayed on the screen, and the program then terminates successfully.

As your programs become more complex, the main() function will often call other functions and coordinate the overall flow of the program. However, it remains the primary entry point from which execution begins.

Topic 2: Basic Input / Output

Concept Overview

C++ programs often communicate with users through streams. The <iostream> library provides the standard input and output objects used in introductory console programs. To use it, at the top of your program, write **#include <iostream>**.

A list of all iostream objects can be found in the table below:

Object	Description
std::cerr	An output stream for error messages
std::clog	An output stream to log program information
std::cin	An input stream that reads keyboard input from the console by default
std::cout	An output stream that writes output to the console by default
std::wcerr	The same as **cerr** but outputs wide char data rather than char data
std::wclog	The same as **clog** but outputs wide char data rather than char data
std::wcin	The same as **cin** but interprets each input character as a wide char
std::wcout	The same as **cout** but outputs wide char data rather than char data

For this introduction, we will only use the **std::cout** and **std::cin** objects. To avoid using the std:: in front of every object, we can use the std namespace. To do that, below the library call, use the following line of code: **using namespace std;**

Key Ideas

- Use #include <iostream> when working with std::cout and std::cin.
- Use variables to store information entered by the user.
- Choose data types carefully: int for whole numbers, double for decimal values, char for a single character, and std::string for text.
- Expressions can combine variables, constants, and arithmetic operators such as +, -, *, and /.
- **Important:** When working with decimal formulas, make at least one operand a floating-point value (for example, 5.0/9.0) so that C++ performs floating-point division.

Example Syntax

```
#include <iostream>
#include <string>

int main()
{
    int age;
    std::string firstName;

    std::cout << "Enter your first name: ";
    std::cin >> firstName;

    std::cout << "Enter your age: ";
    std::cin >> age;

    std::cout << "Hello " << firstName << ", you are " << age << "." << std::endl;
    return 0;
}
```

Exercises

1. Write a program that prints “Hello, World!” to the console. Remember to include the <iostream> library and use the std namespace.

2. Create a program that reads and prints your name. Ask the user to input their name and then print it.

3. Write a program that reads two integers from the user, adds them together, and prints the result.

4. Create a program that calculates and prints the area of a circle. Ask the user to input the radius. The formula for calculating the area of a circle is: A = π*r^2, where π = 3.14 and r is the radius.

5. Write a program that converts temperature from Fahrenheit to Celsius. Ask the user for the temperature in Fahrenheit and print the equivalent temperature in Celsius. The formula for converting from Fahrenheit to Celsius is C = 5/9*(F-32)

Topic 3: if..else if..else statements

Concept Overview

Conditional statements allow a program to choose which instructions to execute. An if statement evaluates a condition. If the condition is true, its block runs. An optional else block runs when the condition is false.

When more than two alternatives are required, else if can be used to test additional conditions in order. Only the first matching branch in an if / else if / else chain is executed.

Conditions commonly use comparison operators such as **==**, **!=**, **<**, **>**, **<=**, and **>=**. **Remember** that **=** assigns a value, while **==** compares two values.

Key Ideas

- A condition evaluates to true or false.
- Braces { } group the statements controlled by each branch.
- Order matters in an else if chain because testing stops after the first true condition.
- The modulo operator % is useful when checking divisibility.

Example Syntax

```
int score = 72;

if (score >= 70)
{
    std::cout << "High mark";
}
else if (score >= 50)
{
    std::cout << "Pass";
}
else
{
    std::cout << "Further practice needed";
}
```

Exercises

6. Write a program that checks if a given number is positive. Ask the user to input a number and print whether it's positive or not.

7. Create a program that determines if a given integer is even or odd. Ask the user for an integer and print the result.

8. Write a program that calculates and prints the maximum of three numbers. Ask the user to input three numbers and determine the maximum value.

Topic 4: Switch...case...default

Concept Overview

A switch statement is useful when one expression must be compared against several fixed values. It can make code clearer than a long chain of if / else if conditions when all branches depend on the same variable.

Each case represents a possible value. The break statement normally ends the selected case so execution does not continue into the next one. The optional default case handles values that do not match any listed case.

In introductory C++, switch is commonly used with integral or enumeration values such as int and char.

Key Ideas

- Use one switch expression and multiple case labels.
- Use break unless deliberate fall-through is intended.
- Use default as a catch-all branch.
- Multiple case labels can share the same code when several input values should lead to the same result.

Example Syntax

```
char option = 'b';

switch (option)
{
    case 'a':
        std::cout << "Option A";
        break;
    case 'b':
        std::cout << "Option B";
        break;
    default:
        std::cout << "Unknown option";
        break;
}
```

Exercises

9. Using switch..case..default, create a program that takes a character from the user and determines if it is a vowel(a, e, i, o, u) or a consonant.

Topic 5: Logical Operators and Precedence

Concept Overview

Logical operators combine or invert conditions. C++ uses:

- **&&** for logical AND;
- **||** for logical OR;
- **!** for logical NOT.

These operators are especially useful when a decision depends on more than one comparison.

Operator precedence determines the order in which parts of an expression are evaluated. For example, arithmetic operations and comparisons are generally evaluated before logical AND/OR. Parentheses should be used whenever they make the intended logic clearer.

C++ uses short-circuit evaluation for && and ||: once the final result is already known, later parts of the expression may not be evaluated.

Key Ideas

- A && B is true only when both A and B are true.
- A || B is true when at least one operand is true.
- !A reverses the truth value of A.
- Parentheses are recommended in compound conditions to improve readability and avoid precedence mistakes.

Example Syntax

```
int value = 25;


if ((value >= 10) && (value <= 30))
{
    std::cout << "Inside the range";
}


bool allowed = (value == 10) || (value == 20);
```

Exercises

10. Write a program that checks if a given number is within the range of 10 to 50 (inclusive). Use logical operators to perform the check.

11. Create a program that checks if a given year is leap year. A leap year is divisible by 4 but not divisible by 100 unless it is also divisible by 400. Use logical operators to determine if the year is a leap year.

Topic 6: for loops

Concept Overview

A for loop repeats a block of code while a condition remains true. It is particularly useful when the number of repetitions is known or can be controlled with a counter.

A typical for loop contains three parts: initialization, condition, and update. The initialization runs once, the condition is checked before every iteration, and the update runs after each iteration.

Loop counters are often declared as integers and updated using ++ or --. The loop body can perform calculations, print values, or accumulate a running result.

Key Ideas

- Initialization sets the starting value of the loop counter.
- The condition decides whether another iteration should run.
- The update changes the counter after each iteration.
- Accumulator variables can be used to build sums, products, and other repeated calculations.

Example Syntax

```
for (int i = 1; i <= 5; ++i)
{
    std::cout << "Iteration " << i << std::endl;
}
```

Exercises

12. Write a program that prints the numbers from 1 to 10 using a for loop.

13. Create a program that calculates and prints the factorial of a number entered by the user, using a for loop.

14. Create a program that generates and prints a multiplication table for a umber entered by the user. Use a for loop to calculate and display the table.

Topic 7: while loops

Concept Overview

A while loop repeats a block of code as long as its condition is true. The condition is checked before the loop body executes, so a while loop may run zero times.

while loops are useful when the exact number of iterations is not known in advance. The loop body normally changes some state that eventually makes the condition false; otherwise, the program may enter an infinite loop.

Any calculation that can be expressed with a for loop can usually also be written with a while loop, although one form may be clearer than the other.

Key Ideas

- The condition is evaluated before every iteration.

- The loop body might never run if the initial condition is false.
- You must update the variables involved in the condition.
- Use while when repetition depends on a changing condition rather than a fixed count.

Example Syntax

```
int i = 1;


while (i <= 5)
{
    std::cout << i << std::endl;
    ++i;
}
```

Exercises

15. Write a program that prints the number from 1 to 10 using a while loop.

16. Create a program that calculates and prints the factorial of a number entered by the user, using a while loop.

Topic 8: do-while loops

Concept Overview

A do-while loop is similar to a while loop, but the condition is checked after the loop body. This guarantees that the body is executed at least once.

This form is useful for input menus, validation routines, and any situation where an action must occur once before the program can decide whether to repeat it.

The syntax ends with a semicolon after the while (condition) line. Forgetting this semicolon is a common beginner error.

Key Ideas

- The loop body runs before the condition is tested.
- A do-while loop therefore executes at least once.
- Update the state used by the condition within the loop body.
- Remember the semicolon after the closing while (condition);.

Example Syntax

```
int i = 1;


do
{
    std::cout << i << std::endl;
    ++i;
}
while (i <= 5);
```

Exercises

17. Write a program that prints the numbers from 1 to 10 using a do-while loop.

18. Create a program that calculates and prints the factorial of a number entered by the user, using a do-while loop.

Topic 9: Nested for loops

Concept Overview

A nested loop is a loop placed inside another loop. For every iteration of the outer loop, the inner loop completes its own set of iterations. Nested loops are useful for grids, tables, matrix processing, and text-based patterns.

When printing patterns, the outer loop often controls the row, while one or more inner loops control the content printed inside that row. Separate counters such as i and j make it easier to distinguish the current row and column.

Because nested loops multiply the amount of repeated work, it is important to understand how many times each loop runs.

Key Ideas

- The inner loop restarts for every outer-loop iteration.
- Use different counter variables for different loop levels.
- Patterns often require separate loops for spaces and visible symbols.
- Indent nested blocks consistently so the program structure remains easy to follow.

Example Syntax

```
for (int row = 1; row <= 3; ++row)
{
    for (int column = 1; column <= 4; ++column)
    {
        std::cout << "*";
    }
    std::cout << std::endl;
}
```

Exercises

19. Write a program that prints a square pattern of asterisks (“*”) with a side length of 5.

20. Create a program that prints a right-angled triangle pattern of numbers from 1 to 5 (Hint: you need to use ++I and ++j).

21. Create a program that prints a diamond pattern of asterisks (“*”) with a height of 5.

Exercise 19 expected output:

```
*****
*****
*****
*****
*****
```

Exercise 20 expected output:

```
1
12
123
1234
12345
```

Exercise 21 expected output:

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

Topic 10: Arrays

Concept Overview

An array stores multiple values of the same data type under one variable name. Each element is accessed using an index. In C++, array indexes begin at 0, so an array with five elements has valid indexes from 0 to 4.

Arrays are useful when several related values must be processed in the same way. A loop can traverse the array to print elements, calculate a sum, find a maximum, or compute an average.

Traditional fixed-size C++ arrays do not automatically check whether an index is valid. Accessing outside the array bounds causes undefined behaviour, so loop conditions must be written carefully.

Key Ideas

- All elements in a traditional array have the same data type.
- Indexing begins at 0.
- Use loops to process all elements efficiently.
- Keep track of the array length and never access an index outside its valid range.

Example Syntax

```
int values[5] = {4, 8, 15, 16, 23};


for (int i = 0; i < 5; ++i)
{
    std::cout << values[i] << std::endl;
}
```

Exercises

22. Create an array of integers with 5 elements and print them.

23. Write a program that calculates and prints the sum of all elements in an array of integers.

24. Write a program that finds and prints the largest element in an array of integers.

25. Create an array of integers and calculate and print the average of its elements.

Topic 11: Functions

Concept Overview

A function is a named block of code that performs a specific task. Functions help divide a program into smaller, reusable parts. They also improve readability by giving meaningful names to pieces of behaviour.

A function may accept parameters, which provide input values to the function. It may also return a value to the caller. The return type appears before the function name; for example, int indicates that the function returns an integer, while bool returns either true or false.

void is used when a function performs an action but does not return a value. Functions should usually be designed to perform one clear responsibility.

Key Ideas

- A function has a return type, name, parameter list, and body.
- Arguments are the actual values supplied when the function is called.
- Parameters are the local variables declared in the function definition.
- Use return to send a value back to the caller when the function has a non-void return type.

Example Syntax

```
int multiply(int a, int b)
{
    return a * b;
}


int main()
{
    int result = multiply(4, 6);
    std::cout << result << std::endl;
    return 0;
}
```

Exercises

26. Create a function that takes two integers as input and returns their sum. Use this function to add two numbers entered by the user and print the results.

27. Write a function (of type integer) that calculates and returns the factorial of a given non-negative integer. Then, use this function to print the factorial of a number entered by the user.

28. Create a function (of type bool) that checks if a given integer is even or odd. The function should return true if the number is even and false if it is odd. Use this function to determine if a number entered by the user is even or odd.

29. Write a function that calculates and returns the sum of all even numbers from 1 to a given positive integer. Use this function to print the sum of even numbers for a number entered by the user.

30. Write a function that prints the sum of all integers inside an array. The function should take the array and its length as parameters.

Topic 12: structs

Concept Overview

A struct is a user-defined data type (similar to int, string, double, etc.) that groups related values, and functions into one object. Unlike an array, the members of a struct can have different data types. For example, one Person object can contain a string name and an integer age.

After defining a struct, variables of that struct type can be created. Members are accessed using the dot operator (.). Structs provide an important first step toward modelling real-world entities and understanding object-oriented programming in C++.

By grouping related data under a meaningful type name, structs make programs easier to organize and understand.

Key Ideas

- Define the structure of the data once, then create multiple instances if needed.
- Struct members may use different data types.
- Access members using the dot operator, such as student.name.
- Structs can be passed to functions and stored in arrays just like many built-in data types.

Example Syntax

```
#include <string>

struct Student
{
    std::string name;
    int year;
};

Student s;
s.name = "Alex";
s.year = 1;
```

Exercises

31. Create a struct called “Person” to store information about a person, including their name (string) and age (integer). Then, write a program that creates an instance of the “Person” struct, assigns values to its members, and then prints out the person’s name and age.

32. Define a struct called “Rectangle” to represent a rectangle’s width and height (both as double values). Then, write a program that calculates and prints the area and perimeter of a rectangle based on user input for its width and height. The formula for calculating the perimeter for a rectangle is P = 2 * (w + h), while the formula for calculating the Area of a rectangle is A = w * h

Lab Completion Checklist:

Before finishing the lab, check that you can do the following without relying on completed solutions:

- Create, compile, and run a C++ source file in Visual Studio Code.
- Use standard input/output and select suitable variable types.
- Write conditions using if/else, switch, comparison operators, and logical operators.
- Choose appropriately between for, while, and do-while loops.
- Use nested loops to create two-dimensional text patterns.
- Store and process multiple values using arrays.
- Decompose a program into reusable functions with parameters and return values.
- Define and use simple structs to group related data.